

嵌入式就业冲刺-重点复盘-总结-补充

《内部资料，拒绝分享，导致面试内卷》

一. C语言阶段

1. 高频出现的关键字篇

1.1 define关键字

重要提醒：笔试的时候，宏的拼写用大写字母！虽然语法不会报错，但会留下不专业的印象。

`#define` 是 C 语言中的一个预处理指令，而不是一个编译时的关键字。

它用于在预处理阶段向源代码中定义宏。

宏是源代码中的一个标识符，被替换为另一个字符串，这在编译之前发生。

下面是关于 `#define` 的一些基本用法和细节：

1.1.1 定义简单的宏

示例：定义一个常量

```
#include <stdio.h>

// 定义一个简单的宏
#define PI 3.14159

int main() {
    double radius = 10.0;
    double area = PI * radius * radius;
    printf("The area of the circle is: %f\n", area);
    return 0;
}
```

这个宏定义使得在代码中每次出现 `PI` 时，预处理器都会将其替换为 `3.14159`。

两个好处（面试过程中可能会问）：

- 批量修改，只要修改 `PI` 后面的 `3.14.159`，那么后续代码中都会被修改，如果没有用宏定义，有多少个地方出现 `3.14159` 就要修改多少次
- 程序可阅读性更高，比如 `setArea(100,200)`; `setArea` 函数假设是获取某个区域，`100,200` 怪怪的，如果 `define WIDTH 100`，`define HEIGHT 200`，那么在函数调用的时候，`setArea(WIDTH, HEIGHT)`; 看着就非常直观。

1.1.2 定义带参数的宏

！！！！！！ 示例代码高频出现笔试中！！！！！！！！！！

示例：定义求平方

```
#include <stdio.h>
// 定义一个带参数的宏
#define SQUARE(x) ((x) * (x))

int main() {
    int num = 5;
    int square = SQUARE(num);
    printf("The square of %d is %d\n", num, square);
    return 0;
}
```

这个宏定义使得 `SQUARE(a)` 会被替换为 `((a) * (a))`。

示例：定义一年多少天

```
#include <stdio.h>

// 定义一个带参数的宏，计算一年的天数
#define DAYS_IN_YEAR(isLeapYear) ((isLeapYear) ? 366 : 365)

int main() {
    int isLeapYear = 1; // 假设这是闰年
    int days = DAYS_IN_YEAR(isLeapYear);
    printf("A leap year has %d days.\n", days);

    isLeapYear = 0; // 假设这是平年
    days = DAYS_IN_YEAR(isLeapYear);
    printf("A common year has %d days.\n", days);

    return 0;
}
```

在这个代码中，`DAYS_IN_YEAR` 宏接受一个参数 `isLeapYear`，如果该参数为真（非零值），则返回366天，表示闰年；如果为假（零值），则返回365天，表示平年。

当你编译并运行这段代码时，它会输出：

```
A leap year has 366 days.
A common year has 365 days.
```

这样，你可以根据输入的参数来判断一年有多少天了。

示例：求圆面积和周长

```
// 定义计算圆的面积的宏
#define CIRCLE_AREA(radius) (3.14159 * (radius) * (radius))

// 定义计算圆的周长的宏
#define CIRCLE_CIRCUMFERENCE(radius) (2 * 3.14159 * (radius))
```

示例：返回小值

宏可以用来替换复杂的代码表达式，简化代码的编写。

```
#define MIN(a, b) ((a) < (b) ? (a) : (b))
int min = MIN(5, 10);
```

示例：宏与函数的比较

宏定义一个加法，也经常考！

```
#include <stdio.h>

// 定义一个宏
#define ADD(a, b) ((a) + (b))

// 定义一个函数
int add(int a, int b) {
    return a + b;
}

int main() {
    int result1 = ADD(3, 4);
    int result2 = add(3, 4);
    printf("Result of macro: %d\n", result1);
    printf("Result of function: %d\n", result2);
    return 0;
}
```

在这个示例中，我们定义了一个 `ADD` 宏和一个 `add` 函数，它们都用来计算两个数的和，并在 `main` 函数中调用。

这些示例展示了 `#define` 在不同场景下的使用方式，包括简单的宏定义、带参数的宏、字符串化宏以及宏与函数的比较。

!!!! 带参数的宏与函数的区别，面试会问，能回答以下三个就够用!!!! :

- 宏在预处理阶段展开，没有类型检查，而函数调用在编译时处理，有类型检查。
- 宏可以处理多行语句，而函数不能直接返回多行语句。
- 宏的参数没有存储空间的概念，而函数的参数在调用时会占用栈空间。

下面是带参数宏定义和函数的区别，以表格形式展示：了解一下就行，主要是上面三点

特性	带参数宏定义	函数
定义	使用 <code>#define</code> 和替换文本定义宏	使用 <code>return</code> 语句和花括号 <code>{}</code> 定义函数
类型检查	没有类型检查，仅文本替换	有类型检查，参数和返回值都有明确类型
作用域	宏在预处理阶段展开，没有实际的代码作用域概念	函数有作用域，可以访问局部变量和参数

特性	带参数宏定义	函数
调用方式	直接使用宏名和参数，如 <code>MACRO_NAME(arg1, arg2)</code>	调用函数名和参数，如 <code>functionName(arg1, arg2)</code>
存储	不占用内存，宏展开后替换为代码	函数体存储在内存中，占用空间
效率	通常比函数调用更快，因为它们在编译前展开，没有调用开销	函数调用有额外的调用和返回开销
调试	调试时看到的是展开后的代码，可能难以追踪原始宏调用	调试时可以轻松定位到函数调用和函数体
副作用	宏展开可能导致意外行为，如重复计算或与运算符优先级相关的问题	函数调用不会导致这类问题
多行语句	可以定义包含多行语句的宏	函数体可以包含多行语句
返回值	宏不返回值，它们只是代码替换	函数可以返回值
内存管理	不涉及内存分配和释放	函数可以使用动态内存分配（如 <code>malloc</code> 、 <code>free</code> ）
重入性	宏可能不是重入的，因为它们可能依赖全局状态或外部变量	函数设计为重入的，不依赖于外部状态

这个表格总结了带参数宏定义和函数在定义、类型检查、作用域、调用方式、存储、效率、调试、副作用、多行语句、返回值和内存管理等方面的区别。选择使用宏还是函数，取决于具体的应用场景和对上述特性的需求。

示例：字符串化运算符：

这个出现不多-但它是陷阱，也许在选择题中

字符串化运算符（`#`）是 C 语言预处理中的一个特性，它用于将宏的参数转换成一个字符串字面量。这个运算符在宏定义中非常有用，尤其是当你需要在编译时将某个标识符或表达式转换成字符串时。

```
#include <stdio.h>

// 这个宏 STRINGIFY 接受一个参数 x，并将其转换成一个字符串。当你使用这个宏时，x 被替换为 #x，
// 预处理器会将 x 视为一个字符串。
#define STRINGIFY(x) #x

int main() {
    const char* name = "Kimi";
    // 直接打印变量的值
    printf("The name is: %s\n", name);
    // 使用宏将变量名转换为字符串，并打印
    printf("The name of the variable is: %s\n", STRINGIFY(name));
    return 0;
}
```

当你编译并运行这段代码时，它会输出：

```
The name is: Kimi
```

```
The name of the variable is: name
```

所以这是个陷阱，我们以为像`define`一个`PI`一样，会输出`3.14159`，以为这里会输出`Kimi`，其实不是!!!

在这个示例中，`STRINGIFY` 宏将 `name` 转换成了字符串 `"name"`，并在 `printf` 函数中使用。

1.1.3 宏的取消定义 (#undef)

宏的取消定义 (`#undef`) 是一个预处理指令，用于取消之前定义的宏。这在某些情况下很有用，比如当你想要重新定义一个宏或者在条件编译中控制宏的定义。

以下是一个包含 `main` 函数的示例代码，展示如何使用 `#undef` 来取消宏的定义：

```
#include <stdio.h>

// 定义一个宏
#define CHEN "very handsome"

int main() {
    // 使用宏
    printf("%s\n", CHEN);

    // 取消宏的定义
    #undef CHEN

    // 尝试使用已取消定义的宏（这将导致编译错误）
    // printf("%s\n", GREETING);

    return 0;
}
```

在这个示例中，我们首先定义了一个名为 `CHEN` 的宏，它包含了一个字符串。在 `main` 函数中，我们使用这个宏来打印一条消息。接着，我们使用 `#undef` 指令取消了 `CHEN` 宏的定义。如果我们尝试再次使用 `CHEN` 宏，预处理器会报错，因为它已经不再定义了。

编译并运行这段代码时，输出将会是：

```
very handsome
```

1.1.4 修饰头文件

假设我们有一个头文件 `myheader.h`，我们希望确保其中的内容只被包含一次：

```
// myheader.h
#ifndef MYHEADER_H
#define MYHEADER_H

// 头文件内容
void myFunction() {
    // 函数实现
}

#endif // MYHEADER_H
```

在这个头文件中，`#ifndef MYHEADER_H` 检查 `MYHEADER_H` 宏是否未定义，如果是，则定义它，并包含头文件的内容。`#endif` 标记了条件编译块的结束。

现在，让我们看看如何在一个 C 程序中使用这个头文件：

```
#include <stdio.h>

// 假设这是我们的头文件
#ifndef MYHEADER_H
#define MYHEADER_H

void myFunction() {
    printf("Hello from myFunction!\n");
}

#endif // MYHEADER_H

int main() {
    myFunction(); // 调用头文件中定义的函数
    return 0;
}
```

在这个程序中，当我们包含 `myheader.h` 时，`#ifndef`、`#define` 和 `#endif` 确保了即使 `myheader.h` 被多次包含（例如，通过多个不同的源文件），`myFunction` 也只会被定义一次。这样可以避免链接错误和重复定义的问题。

编译并运行这段代码时，输出将会是：

```
Hello from myFunction!
```

这种使用 `#ifndef`、`#define` 和 `#endif` 的技术被称为“包含卫士”或“头文件卫士”，是管理头文件内容被多次包含的一种标准方法。

1.2 typedef关键字-别名作用

`typedef` 是 C 语言中的一个关键字，用于为数据类型定义一个新的名称，称为别名（alias）。使用 `typedef` 可以增加代码的可读性，使得类型定义更加清晰和简洁。

1.2.1 为基本数据类型定义别名

```
#include <stdio.h>

// 为基本数据类型定义别名
typedef unsigned char Byte;           // 8位无符号整数
typedef char* String;                // 字符串类型
typedef float Real;                  // 单精度浮点数
typedef int Bool;                    // 布尔类型，用整型表示
typedef unsigned long ulong;         // 无符号长整数

// 为布尔类型定义两个常量
#define TRUE 1
```

```

#define FALSE 0

int main() {
    // 使用别名声明变量
    Byte byteValue = 0x0F;
    String str = "Hello, world!";
    Real realValue = 3.14159f;
    Bool boolValue = TRUE;
    ulong ulongValue = 1234567890UL;

    // 打印变量值
    printf("Byte value: %u\n", byteValue);
    printf("String: %s\n", str);
    printf("Real value: %f\n", realValue);
    printf("Boolean value: %s\n", boolValue ? "TRUE" : "FALSE");
    printf("Unsigned long value: %lu\n", ulongValue);

    return 0;
}

```

1.2.2 为复杂的数据类型定义别名

以下是使用 `typedef` 为复杂数据类型（如结构体和联合体）定义别名，并在 `main` 函数中使用这些别名的示例代码：

```

#include <stdio.h>

// 为结构体定义别名
typedef struct {
    int id;
    float salary;
    char name[50];
} Employee;

// 为联合体定义别名
typedef union {
    int intValue;
    float floatValue;
    char charValue;
} Data;

int main() {
    // 使用结构体别名声明变量
    Employee emp = {1, 50000.0f, "John Doe"};

    // 使用联合体别名声明变量
    Data data;
    data.intValue = 10;

    // 打印结构体变量的值
    printf("Employee ID: %d\n", emp.id);
    printf("Employee Salary: %.2f\n", emp.salary);
    printf("Employee Name: %s\n", emp.name);
}

```

```

// 打印联合体变量的值
printf("Data as int: %d\n", data.intValue);

// 重新赋值并打印联合体变量的值
data.floatValue = 3.14f;
printf("Data as float: %.2f\n", data.floatValue);

// 注意：由于联合体的特性，打印 intValue 可能会得到意外的结果
// printf("Data as int after float assignment: %d\n", data.intValue);

return 0;
}

```

之前我们可能是这么写的

```

#include <stdio.h>

// 为结构体定义别名
struct Person{
    int id;
    float salary;
    char name[50];
};

int main() {
    struct Person P1 = {1, 50000.0f, "John Doe"};;
    struct Person P2; //在定义的时候要加struct关键字

    // 使用结构体别名声明变量,就不用加struct关键字了
    // Employee emp = {1, 50000.0f, "John Doe"};
    return 0;
}

```

1.2.3 为数组类型定义别名

这个用的不多，有点怪，可以简单涉猎下

这里有一个使用 `typedef` 为数组类型定义别名，并在 `main` 函数中使用这些别名的示例代码：

```

#include <stdio.h>

// 为数组类型定义别名
typedef char CharArray[100]; // 固定大小的字符数组
typedef int IntArray[10];    // 固定大小的整数数组

int main() {
    // 使用别名声明数组变量
    CharArray name = "Kimi";
    IntArray scores;

    // 初始化数组
    for (int i = 0; i < 10; ++i) {

```

```

        scores[i] = i * 10;
    }

    // 打印字符数组
    printf("CharArray: %s\n", name);

    // 打印整数数组
    printf("IntArray contains: ");
    for (int i = 0; i < 10; ++i) {
        printf("%d ", scores[i]);
    }
    printf("\n");

    return 0;
}

```

在这个示例中，我们定义了两个数组类型的别名：

- `CharArray` 作为包含100个字符的数组的别名。
- `IntArray` 作为包含10个整数的数组的别名。

在 `main` 函数中，我们使用这些别名来声明 `CharArray` 类型的变量 `name` 和 `IntArray` 类型的变量 `scores`。然后，我们为 `scores` 数组中的每个元素赋值，并使用 `printf` 函数打印 `name` 和 `scores` 数组的内容。

这个示例展示了如何使用 `typedef` 为数组类型创建更简洁的别名，使得数组的声明更加方便和清晰。

1.2.4 为指针类型定义别名

以下是使用 `typedef` 为指针类型定义别名，并在 `main` 函数中使用这些别名的示例代码：

```

#include <stdio.h>
#include <stdlib.h>

// 为指针类型定义别名
typedef char* PChar;      // 指向字符的指针
typedef int* PInt;        // 指向整数的指针
typedef float* PFloat;    // 指向浮点数的指针

int main() {
    // 使用别名声明指针变量
    PChar str = malloc(100 * sizeof(char)); // 分配内存并初始化字符串
    PInt num = malloc(sizeof(int));          // 分配内存以存储整数
    PFloat value = malloc(sizeof(float));    // 分配内存以存储浮点数

    // 初始化指针指向的值
    *str = 'H'; // C风格字符串以'\0'结尾，这里仅为示例，不进行完整字符串赋值
    *str = '\0'; // 确保字符串正确终止
    *num = 42;
    *value = 3.14f;

    // 打印指针指向的值
    printf("Pointer to char points to: %s\n", str);
    printf("Pointer to int points to: %d\n", *num);
}

```

```

    printf("Pointer to float points to: %.2f\n", *value);

    // 释放内存
    free(str);
    free(num);
    free(value);

    return 0;
}

```

1.2.5 为函数指针类型定义别名

为函数指针类型定义别名时，我们使用 `typedef` 关键字来创建一个新的类型名，这样可以使得函数指针类型的声明更加简洁和清晰。以下是一个示例代码，展示如何为函数指针类型定义别名，并在 `main` 函数中使用这个别名：

```

#include <stdio.h>

// 为函数指针类型定义别名
// 这个函数指针接受两个int参数并返回一个int
typedef int (*IntFunctionPtr)(int, int);

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    // 使用别名声明函数指针变量
    IntFunctionPtr funcPtr;

    // 给函数指针变量赋值
    funcPtr = add; // 指向add函数
    printf("10 + 5 = %d\n", funcPtr(10, 5));

    funcPtr = subtract; // 指向subtract函数
    printf("10 - 5 = %d\n", funcPtr(10, 5));

    return 0;
}

```

在这个示例中，我们定义了一个名为 `IntFunctionPtr` 的别名，它是一个函数指针类型，指向接受两个 `int` 参数并返回一个 `int` 的函数。然后我们定义了两个简单的函数 `add` 和 `subtract`，它们都符合这个签名。

在 `main` 函数中，我们声明了一个 `IntFunctionPtr` 类型的变量 `funcPtr`，并分别将其指向 `add` 和 `subtract` 函数。然后通过函数指针调用这些函数，并打印结果。

这个示例展示了如何使用 `typedef` 为函数指针类型定义别名，这样做可以使得代码更加简洁，特别是在处理复杂的函数指针时。使用别名可以避免每次都写出完整的函数指针类型，从而使代码更加易读。

以下是函数指针不用别名的方式，回忆一下：

```

#include <stdio.h>

// 定义两个符合特定签名的函数
int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    // 声明函数指针变量，但不使用别名
    int (*funcPtr)(int, int);

    // 给函数指针变量赋值
    funcPtr = add; // 指向add函数
    printf("10 + 5 = %d\n", funcPtr(10, 5));

    funcPtr = subtract; // 指向subtract函数
    printf("10 - 5 = %d\n", funcPtr(10, 5));

    return 0;
}

```

1.2.6 特点总结

- `typedef` 创建的是真实的类型别名，它不仅用于代码的阅读，而且在编译时也会将别名替换为实际的类型。
- `typedef` 定义的别名在编译时会进行类型检查，这是它与宏定义的主要区别之一。
- `typedef` 可以减少代码中的复杂性，特别是在处理嵌套的数据结构时。
- `typedef` 可以提高代码的可移植性，因为你可以为特定平台定义特定的类型别名。

示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `typedef`：

```

#include <stdio.h>

// 使用 typedef 为 int 定义别名
typedef int Length;

// 使用 typedef 为结构体定义别名
typedef struct {
    Length length;
    Length width;
} Rectangle;

int main() {
    Rectangle rect;
    rect.length = 10;
    rect.width = 5;

    // 使用 typedef 定义的别名
}

```

```

    Length perimeter = 2 * (rect.length + rect.width);
    printf("The perimeter of the rectangle is: %d\n", perimeter);
    return 0;
}

```

在这个示例中，我们为 `int` 类型定义了一个别名 `Length`，并为一个结构体定义了一个别名 `Rectangle`。在 `main` 函数中，我们使用这些别名来声明变量和计算矩形的周长。

使用 `typedef` 可以让代码更加简洁和易于理解，尤其是在处理复杂的数据结构时。它也有助于在不同的平台之间保持代码的一致性，因为你可以为特定的数据类型定义统一的别名。

1.3 extern关键字

C语言中的 `extern` 关键字用于**声明**一个变量或函数是在另一个文件或编译单元中**定义**的。

<!! 什么是声明，什么是定义，本节后面讲，面试也会作怪问这种问题!! >

优点：或者说为什么要用`extern`，和头文件的区别是什么：

`extern` 关键字的使用可以减少代码重复，提高模块化，使得程序的各个部分可以更容易地共享资源。

以下是 `extern` 关键字的一些关键点：

1.3.1 全局变量的外部链接

当一个全局变量被声明为 `extern` 时，它允许当前文件访问在其他文件中定义的全局变量。

```

// file1.c
int global_var = 10;

// file2.c 当前文件file2.c访问其他文件file1.c中定义的全局变量
extern int global_var; // 声明global_var在其他文件定义
printf("%d", global_var); // 可以访问file1.c中定义的global_var，这里输出的结果是file1.c
                          中的10

```

1.3.2 函数的外部链接

类似地，`extern` 也用于声明函数，允许当前文件调用在其他文件中定义的函数。

```

// file1.c
#include <stdio.h>
void function() {
    // 函数实现
    printf("c1c handsome\n");
}

// file2.c
extern void function(); // 声明function在其他文件定义
function(); // 调用file1.c中定义的function,c1c handsome

```

1.3.3 头文件中的使用

`extern` 经常在头文件中使用，以声明全局变量和函数，这样多个文件就可以包含同一个头文件并访问这些全局资源，而不需要在每个文件中都重新定义它们。

```
// header.h
extern int global_var; // 声明变量
extern void function(); // 声明函数

// file1.c
#include "header.h"
int global_var = 10; // 定义变量
void function() {    // 定义并实现函数

}

// file2.c
#include "header.h"
function(); // 调用函数
```

默认的外部链接，上述案例中，不用`extern`也是可以的，因为

在C语言中，如果没有特别指定存储类，全局变量和全局函数默认具有外部链接。使用 `extern` 关键字可以明确指出这一点，尤其是在跨文件共享全局资源时。

1.3.4 声明和定义的区别

在编程语言中，特别是在C和C++中，"声明" (Declaration) 和"定义" (Definition) 是两个不同的概念，它们在代码中扮演着不同的角色：

声明 (Declaration) :

- 声明变量时，它不会分配内存空间；
- 声明函数时，它提供了函数的名称、返回类型和参数列表，但不包含函数体。
- 声明可以多次出现在程序中
- 声明通常用于头文件中，通过 `extern` 关键字来实现跨文件的引用。

例如：

```
int globalVar; // 变量声明
void function(); // 函数声明
```

定义 (Definition) :

- 定义变量时，它在内存中分配空间，并可能初始化。
- 定义函数时，它包含了函数的实现（函数体）。
- 每个标识符在程序中只能有一个定义。
- 定义通常出现在源文件（`.c` 或 `.cpp` 文件）中。

例如：

```
int globalVar = 42; // 变量定义
void function() {
    // 函数体
} // 函数定义
```

区别:

- **作用域:** 声明可以出现在头文件中，供多个源文件包含，而定义通常只出现在一个源文件中。
- **内存分配:** 声明不分配内存，定义变量时分配内存。
- **实现细节:** 声明不包含实现细节，定义包含实现细节，如函数体。
- **数量限制:** 一个标识符可以有多个声明，但只能有一次定义。
- **编译器行为:** 声明让编译器知道标识符的存在和类型，定义则让编译器生成相应的代码。

理解声明和定义的区别对于编写模块化和可重用的代码非常重要，尤其是在大型项目中，它有助于避免命名冲突并确保代码的正确链接。

1.4. static关键字

1.4.1 修饰局部变量

在函数内部声明的静态变量具有以下特性：

- **生命周期:** 静态变量的生命周期从程序开始执行到程序结束，它们在整个程序执行期间都存在，而不是在函数调用期间创建和销毁。
- **可见性:** 静态变量的作用域仅限于包含它们的函数内部，即它们是局部变量，但其值在函数调用之间保持不变。
- **初次初始化:** 静态变量只在第一次进入其定义的函数时进行初始化，并且仅执行一次。

```
#include <stdio.h>
void test()
{
    static int m = 0;
    m = m + 1;
    printf("%d", m);
}

int main()
{
    int n = 0;
    while (n < 10)
    {
        test();
        n++;
    }
}
```

输出结果:

12345678910

1.4.2 修饰全局变量

在函数外部（全局范围）声明的静态变量，又叫**全局静态变量**具有以下特性：

- 生命周期：静态全局变量的生命周期与程序的执行周期相同。
- 可见性：静态全局变量的作用域限于声明它们的源文件，不能被其他源文件使用。

创建一个源文件：a.c

```
int g_val=2024;//全局变量
```

创建一个源文件b.c，引用全局变量：全局变量的作用域是整个工程

```
#include <stdio.h>
extern int g_val;//extern 用来声明外部命令，可以调用到a.c的 g_val=2024
int main()
{
    printf("%d\n", g_val);
    return 0;
}
//两个文件一起编译：gcc a.c b.c ， 程序输出结果是2024
```

static可以修饰全局变量，在a.c 加上static看一下有什么不同：

```
static int g_val = 2024;//全局变量
```

两个文件一起编译：gcc a.c b.c

加上之后,程序编译报错, 被static修饰的全局变量可见性只在声明它的源文件，其他源文件无法使用。

报错：

```
G:\10月份技术方案>gcc a.c b.c
C:\Users\asus\AppData\Local\Temp\ccpByz5S.o:b.c:(.rdata$.refptr.g_val[.refptr.g_val]+0x0): undefined reference to `g_val'
collect2.exe: error: ld returned 1 exit status
```

```
C:\Users\asus\AppData\Local\Temp\ccpByz5S.o:b.c:
(.rdata$.refptr.g_val[.refptr.g_val]+0x0): undefined reference to `g_val'
collect2.exe: error: ld returned 1 exit status
```

1.4.3 static修饰函数

使用'static'关键字声明的函数是静态函数。静态函数具有以下特性：

- 可见性：静态函数的作用域限于声明它们的源文件，不能被其他源文件调用。
- 隐藏性：将函数声明为静态的，可以将其隐藏在当前源文件中，以防止与其他源文件中具有相同名称的函数发生冲突。

源文件a.c中创建个函数名为add：

代码如下：

```
static int add(int x, int y)
{
    int z = x + y;
    return z;
}
```

主文件b.c如下：

```
#include <stdio.h>

extern add(int x, int y);
int main()
{
    int a = 2;
    int b = 3;
    int c = add(a, b);
    printf("%d\n", c);
    return 0;
}
```

其实一个函数本身具有外部链接属性 被static修饰后外部链接属性变成了内部链接属性，只能在源文件a.c 内部使用了，其他源文件无法使用，使用上感觉作用域变小，我们可以发现 static 修饰全局变量和修饰函数用法一样。

编译后发现：

```
G:\10月份技术方案>gcc a.c b.c
C:\Users\asus\AppData\Local\Temp\ccqscgeS.o:b.c:(.text+0x24): undefined reference to `add'
collect2.exe: error: ld returned 1 exit status
```

```
C:\Users\asus\AppData\Local\Temp\ccqscgeS.o:b.c:(.text+0x24): undefined reference
to `add'
collect2.exe: error: ld returned 1 exit status
```

1.4.4 static修饰的变量存放在哪里

能回答的上这个，面试好感上升，正常面试者只会零散回答前面3个

static修饰的变量存放在静态数据区。



在C语言中，static变量

static修饰的变量	bss段	数据段
未显式初始化的static局部变量	bss段	
未显式初始化的static全局变量	bss段	
初始化为0的static局部变量	bss段	
初始化为0的static全局变量	bss段	
显式初始化的static全局变量		数据段
显式初始化的static局部变量	bss段 显式初始化的 static 局部变量实际上存储在BSS段，因为即使它们被显式初始化，它们的存储位置并没有改变。然而，它们的初始化时机和行为与其他BSS段变量不同。static 局部变量的初始化发生在第一次进入包含它们的函数时，而不是在main函数执行之前。此外，它们的生命周期持续到程序结束，而不是在函数调用结束后释放。 《请注意，static 局部变量的存储位置在不同编译器或不同优化级别下可能会有所不同，但它们通常在BSS段，并且初始化时机是在第一次函数调用时。》	

在程序启动时，BSS段会自动被操作系统或加载器初始化为零值。

顺便说说对象的存储，可分为三类：静态存储（static storage）；自动存储（automatic storage）；动态分配存储（allocated or dynamic storage）。

对于自动存储则对应的是栈（stack），动态分配存储对应的是堆（heap）；静态存储可分为.bss/.data/.rodata等数据段（section）。

在程序执行中把初始值为零或者是未设初始值的变量放在.bss段中。

1.5 const关键字

`const` 关键字在 C 语言中有多个作用，主要用于定义常量

保证变量的值在初始化后不可更改，说白了，变量的值不能被修改，就是常量了。

总的来说，`const`关键字在C语言中提供了一种机制来声明只读变量和指针，从而提高了代码的可读性、可维护性和安全性。

通过正确地理解和使用`const`，你可以确保某些数据在程序运行时不会被意外修改。

以下是 `const` 关键字的一些主要作用和相应的示例代码：

1.5.1 定义常量

```
#include <stdio.h>

int main() {
    const int MAX_USERS = 100; // 定义常量，值不可更改
    printf("Maximum number of users: %d\n", MAX_USERS);
    return 0;
}
```

在这个示例中，`MAX_USERS` 被定义为一个常量，其值在程序运行期间不能被修改。

1.5.2 修饰函数参数

```
#include <stdio.h>

void printMessage(const char* message) {
    // 函数内部不能修改 message 的值
    printf("%s\n", message);
}

int main() {
    const char* greeting = "Hello, world!";
    printMessage(greeting);
    return 0;
}
```

在这个示例中，`printMessage` 函数接受一个 `const` 参数，表明在函数内部不能修改 `message` 的值。

1.5.3 修饰函数返回值

```
#include <stdio.h>

const char* getAppName() {
    return "MyApp"; // 返回一个指向常量字符串的指针
}

int main() {
    const char* appName = getAppName();
    printf("Application Name: %s\n", appName);
    return 0;
}
```

在这个示例中，`getAppName` 函数返回一个指向常量字符串的指针，这意味着通过 `appName` 指针不能修改字符串的内容。

1.5.4 修饰指针

```
#include <stdio.h>

int main() {
    const int num = 42;
    int const *ptr = &num; // 指向常量的指针
    printf("%d\n", *ptr);
    // *ptr = 10; // 错误：不能通过 ptr 修改 num 的值
    return 0;
}
```

在这个示例中，`ptr` 是一个指向常量的指针，意味着不能通过 `ptr` 来修改它所指向的值。

1.5.5 修饰数组

```
#include <stdio.h>

int main() {
    const int numbers[] = {1, 2, 3, 4, 5};
    int length = sizeof(numbers) / sizeof(numbers[0]);
    for (int i = 0; i < length; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

在这个示例中，`numbers` 是一个常量数组，意味着数组中的元素在程序运行期间不能被修改。

`const` 关键字的使用可以提高程序的安全性和可读性，它告诉编译器和程序员哪些变量是不应该被修改的。

1.6 sizeof关键字

`sizeof` 是 C 语言中的一个编译时运算符，注意哦，不是函数，像函数，不是函数！！面试有可能会问。

用于确定变量或类型在内存中的大小（以字节为单位）。`sizeof` 可以用于基本数据类型、复合数据类型（如结构体和联合体）以及指针类型。

以下是 `sizeof` 的一些常见用法和特点：

1.6.1 基本用法

1. 获取基本数据类型的大小：

```
printf("Size of char: %zu\n", sizeof(char));
printf("Size of int: %zu\n", sizeof(int));
printf("Size of float: %zu\n", sizeof(float));
```

2. 获取复合数据类型的大小：

```
struct MyStruct {
    int a;
    float b;
    char c;
};
printf("Size of MyStruct: %zu\n", sizeof(struct MyStruct));
```

3. 获取指针类型的大小：

```
int var = 10;
printf("Size of int pointer: %zu\n", sizeof(&var));
```

1.6.2 特点

- `sizeof` 返回的是类型的存储大小，不是类型中元素的数量。
- `sizeof` 可以用于任何数据类型，包括原始类型（如 `int`、`float`）和复合类型（如 `struct`、`union`）。
- `sizeof` 的结果是一个常量，因此可以在编译时计算。
- `sizeof` 可以用于函数参数，但不能用在运行时的变量或表达式上。

1.6.3 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `sizeof`：

```
#include <stdio.h>

int main() {
    int intVar = 10;
    float floatVar = 3.14f;
    double doubleVar = 3.14159;
    char charVar = 'A';
    struct {
        int a;
```

```

    float b;
    char c;
} structVar;

printf("Size of int: %zu\n", sizeof(intVar));
printf("Size of float: %zu\n", sizeof(floatVar));
printf("Size of double: %zu\n", sizeof(doubleVar));
printf("Size of char: %zu\n", sizeof(charVar));
printf("Size of struct: %zu\n", sizeof(structVar));

return 0;
}

```

在这个示例中，我们声明了几种不同类型的变量，并使用 `sizeof` 运算符来获取它们的大小。`%zu` 是 `printf` 的格式说明符，用于打印 `size_t` 类型的值，这是 `sizeof` 运算符的结果类型。

`sizeof` 是一个非常有用的工具，可以帮助你了解数据类型在内存中的布局，对于内存管理、数组操作和数据对齐等场景非常有用。

1.6.4 sizeof和strlen区别

很重要，新手非常非常非常非常非常经常犯下的错误，特别是编程过程中

这段代码提供了一个很好的例子来说明 `sizeof` 和 `strlen` 之间的区别。让我们来分析这段代码并分析输出。

特别注意 `str2[100]!!!`

```

#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, world!";
    char str2[100] = "Hello, world!";
    char *ptr = "Hello";

    // 使用 sizeof 获取类型大小
    printf("Size of int: %zu\n", sizeof(int));
    printf("Size of str: %zu\n", sizeof(str)); // 注意: sizeof 得到的是整个数组的大小
    printf("Size of str2: %zu\n", sizeof(str2)); // 注意: sizeof 得到的是整个数组的大
    小
    // printf("Length of str2: %zu\n", strlen(str2)); // 特别注意这里的strlen(str2)是
    多少!!!
    printf("Size of ptr: %zu\n", sizeof(ptr)); // sizeof 得到的是指针的大小，不是字符
    串的长度

    // 使用 strlen 获取字符串长度
    printf("Length of str: %zu\n", strlen(str));
    printf("Length of str2: %zu\n", strlen(str2));
    printf("Length of ptr: %zu\n", strlen(ptr));

    return 0;
}

```

输出

Size of int: 这将输出 `int` 类型的大小，通常在大多数系统上是 4 字节。

```
Size of int: 4
```

Size of str: 这将输出数组 `str` 的大小。`str` 包含字符串 "Hello, world!"，后面还有一个隐含的空字符 `\0`，共 14 字节。

```
Size of str: 14
```

Size of str2: 这将输出数组 `str2` 的大小，它被显式地声明为大小为 100 的数组。

```
Size of str2: 100
```

Size of ptr: 这将输出指针 `ptr` 的大小，而不是它指向的字符串的长度。在 32 位系统上，指针通常是 4 字节；在 64 位系统上，通常是 8 字节。

```
Size of ptr: 8 (在 64 位系统上)
```

Length of str: 这将输出字符串 `str` 的长度，不包括结尾的空字符 `\0`。字符串 "Hello, world!" 的长度是 13。

```
Length of str: 13
```

Length of ptr: 这将输出字符串 `ptr` 指向的 "Hello" 的长度，不包括结尾的空字符 `\0`。字符串 "Hello" 的长度是 5。

```
Length of ptr: 5
```

区别阐述

- `sizeof`: 用于获取变量或类型在内存中的大小。它是一个编译时运算符，可以用于任何数据类型，包括数组和指针。`sizeof` 对于数组，返回整个数组的大小；对于指针，返回指针本身的大小，而不是指针指向的内容的大小。
- `strlen`: 用于计算以空字符 `\0` 结尾的字符串的长度。它是一个运行时函数，需要一个指向字符数组的指针，并返回字符串的长度，不包括结尾的空字符。`strlen` 只能用于以 `\0` 结尾的字符串，不能用于一般的字符数组。

通过这个代码示例，我们可以清楚地看到 `sizeof` 和 `strlen` 的不同用途和行为。`sizeof` 提供了关于数据类型大小的信息，而 `strlen` 提供了关于字符串内容长度的信息。

1.7 register关键字

不多见，可以了解一下!

在 C 语言中，`register` 关键字是一个建议性的指示符，用于建议编译器将变量存储在 CPU 的寄存器中，而不是 RAM 中。使用 `register` 关键字的目的是为了优化程序性能，因为访问寄存器比访问内存要快得多。

1.7.1 基本用法

```
register int count;
```

在这个例子中，`count` 被声明为一个 `register` 类型的整型变量，这意味着编译器被建议将这个变量存储在寄存器中。

1.7.2 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `register` 关键字：

```
#include <stdio.h>

int main() {
    register int i; // 建议编译器将 i 存储在寄存器中

    for (i = 0; i < 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

在这个示例中，变量 `i` 被声明为 `register` 类型，建议编译器在循环中使用寄存器来存储 `i` 的值，以提高循环的执行速度。

尽管 `register` 关键字在某些情况下可以提供性能优化，但在现代编程实践中，它的使用越来越少，因为编译器的自动优化通常更有效。此外，`register` 的使用可能会限制变量的可见性和调试能力。

1.8 volatile关键字

重要，经常考！

以下那段话，可以背诵或者阐述大概意思，面试会问

`volatile` 是 C 语言中的一个关键字，用于告知编译器该变量可能被程序之外的因素（例如硬件或其他线程）以不可预测的方式改变。使用 `volatile` 关键字的变量，编译器将不会对该变量进行优化，每次访问这个变量时都会从它的存储地址中重新读取值，而不是使用寄存器中的值或缓存的值。

1.8.1 基本用法

```
volatile int flag;
```

在这个例子中，`flag` 被声明为一个 `volatile` 变量，这意味着每次对 `flag` 的访问都应该是直接从它的地址读取，而不是从寄存器或缓存中读取。

1.8.2 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `volatile` 关键字：

```
#include <stdio.h>

// 假设这是由硬件或其他线程修改的全局变量
volatile int globalFlag = 0;

void interruptServiceRoutine() {
    // 模拟硬件中断，改变全局变量的值
    globalFlag = 1;
}

int main() {
    // 模拟等待中断的过程
    while (globalFlag == 0) {
        // 循环检查全局标志是否被设置
    }

    // 一旦标志被设置，执行相应的操作
    printf("Interrupt service routine has been called.\n");

    return 0;
}
```

在这个示例中，`globalFlag` 是一个 `volatile` 变量，它可能被中断服务程序 `interruptServiceRoutine` 改变。在 `main` 函数中，程序循环检查 `globalFlag` 的值，一旦它被设置，就执行相应的操作。

使用 `volatile` 关键字是处理硬件寄存器或多线程环境中变量时的一个重要机制，它确保了程序能够正确响应外部变化。然而，`volatile` 不应该被滥用，因为它不会提供线程安全或同步保证。在需要同步多个线程对共享资源的访问时，应该使用适当的同步机制，如互斥锁（mutexes）或原子操作。

1.9 enum关键字

`enum` 是 C 语言中的一个关键字，用于定义枚举类型，这是一种用户定义的类型，它允许为整型常量赋予有意义的名称。枚举是一种强类型的数据类型，可以提高代码的可读性和可维护性。

1.9.1 基本用法

1. 定义枚举类型：

```
enum Color { RED, GREEN, BLUE };
```

这里定义了一个名为 `Color` 的枚举类型，它包含三个枚举常量：`RED`、`GREEN` 和 `BLUE`。

2. 定义枚举类型并指定底层类型：

```
enum Color : unsigned int { RED = 10, GREEN = 20, BLUE = 30 };
```

在这个例子中，枚举 `Color` 被指定了底层类型 `unsigned int`，并且为每个枚举常量显式赋值。

1.9.2 特点和注意事项

1. **默认值**：如果枚举常量没有显式赋值，它们将从 0 开始自动赋值，每个后面的枚举常量值比前一个值大 1。
2. **底层类型**：枚举类型有一个底层类型，通常是 `int`，但也可以显式指定为其他整型类型，如 `unsigned int`、`long` 等。
3. **存储大小**：枚举类型变量的存储大小取决于其底层类型。
4. **类型安全**：枚举提供了类型安全的方式，限制变量只能取特定的几个值之一。
5. **跨平台兼容性**：枚举常量在不同平台上的底层表示可能不同，因此不应依赖于特定的底层值。

1.9.3 示例代码

以下是一个包含 `main` 函数的示例代码，展示如何使用 `enum` 关键字：

```
#include <stdio.h>

// 定义一个枚举类型表示星期几
enum Weekday { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };

int main() {
    enum Weekday today = TUESDAY;
    printf("Today is %d\n", today);

    switch (today) {
        case SUNDAY:
            printf("Sunday is the first day of the week.\n");
            break;
        case SATURDAY:
            printf("Saturday is the last day of the week.\n");
            break;
        default:
            printf("Today is a weekday.\n");
    }

    return 0;
}
```

```
1  #include <stdio.h>
2
3  // 定义一个枚举类型表示星期几
4  enum Weekday { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
5
6  int main() {
7      enum Weekday today = TUESDAY;
8      printf("Today is %d\n", today);
9
10     switch (today) {
11         case SUNDAY:
12             printf("Sunday is the first day of the week.\n");
13             break;
14         case SATURDAY:
15             printf("Saturday is the last day of the week.\n");
16             break;
17         default:
18             printf("Today is a weekday.\n");
19     }
20
21     return 0;
22 }
```

管理员: 命令提示符

G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Today is 2
Today is a weekday.
G:\10月份技术方案>
运行结果

在这个示例中，我们定义了一个名为 `weekday` 的枚举类型来表示一周的七天。在 `main` 函数中，我们声明了一个 `weekday` 类型的变量 `today` 并给它赋值为 `TUESDAY`，然后使用 `switch` 语句根据 `today` 的值打印相应的信息。

使用 `enum` 可以使得代码更加清晰和易于理解，尤其是在处理一组有限的、相关的常量时。

1.10 auto关键字

这部分简单了解一下。

在 C 语言的历史中，`auto` 关键字最初用于声明自动存储周期的局部变量。然而，在 C99 标准之后，`auto` 的这一用法变得不必要，因为自动存储周期成了局部变量的默认行为。因此，在现代 C 语言编程中，`auto` 关键字的使用越来越少，它通常不被用于声明变量。

1.10.1 基本用法（历史用法）

在 C89/C90 标准中，`auto` 关键字用于声明具有自动存储周期的局部变量，这意味着变量的生命周期仅限于其定义的代码块，并且在函数调用结束后会被销毁。

```
auto int localVar = 10; // 声明一个自动存储周期的整型变量
```

1.10.2 现代用法

在 C99 及以后的版本中，所有的局部变量默认具有自动存储周期，因此 `auto` 关键字变得多余，可以省略。

```
int localVar = 10; // auto 可以省略，这是默认行为
```

1.10.3 C++ 中的 `auto` 关键字

在 C++ 中，`auto` 关键字有一个更现代的用法，它用于自动类型推断。编译器会根据变量的初始值自动确定其类型。

```
auto x = 10;           // x 的类型被推断为 int
auto y = 3.14;         // y 的类型被推断为 double
auto z = "hello";      // z 的类型被推断为 const char*
```

在 C++ 中，`auto` 关键字非常有用，特别是在处理复杂的数据类型或泛型编程时，它可以简化代码并减少打字量。

1.10.4 C11 中的 `auto` 关键字

在 C11 标准中，`auto` 关键字与 `sizeof` 运算符一起使用时，可以用于避免类型转换。具体来说，当你有一个复杂的类型，比如一个数组或者一个结构体，并且你想要获取这个类型的一个元素或者成员的大小时，你可以使用 `auto` 来简化这个过程。

以下是包含 `main` 函数的示例代码，展示如何使用 `auto` 与 `sizeof`：

```
#include <stdio.h>

int main() {
    // 假设有一个结构体
    struct myStruct{
        char a;
        int b;
        double c;
    };

    // 使用 auto 和 sizeof 获取结构体成员的大小
    auto sizeofChar = sizeof(myStruct.a);
    auto sizeofInt = sizeof(myStruct.b);
    auto sizeofDouble = sizeof(myStruct.c);

    // 打印结果
    printf("Size of char: %zu\n", sizeofChar);
    printf("Size of int: %zu\n", sizeofInt);
    printf("Size of double: %zu\n", sizeofDouble);

    return 0;
}

//关于%zu的说明
/*
```

`%zu` 告诉 `printf` 函数期待一个 `size_t` 类型的参数，并且按照适当的方式格式化输出这个值。如果不使用 `%zu` 而使用 `%d` 或 `%u`，那么在 64 位系统上可能无法正确显示大于 4GB 的数组大小，因为 `%d` 和 `%u` 分别对应 `int` 和 `unsigned int` 类型，它们可能不足以存储 `size_t` 类型的所有可能值。

*/

在这个示例中，我们定义了一个结构体 `myStruct`，并且使用 `auto` 与 `sizeof` 运算符来获取结构体中每个成员的大小。`auto` 在这里用于自动推断 `sizeof` 运算符的结果类型，即 `size_t` 类型，这样就避免了显式地写出 `size_t` 类型。

这种用法在处理复杂的数据结构或者数组时特别有用，因为它可以减少代码的复杂性，并且避免类型转换的错误。通过使用 `auto`，编译器会自动处理类型转换，使得代码更加简洁和安全。

总的来说，`auto` 关键字在现代 C 语言中的使用较少，而在 C++ 中则更为常见，用于自动类型推断。在 C11 中，`auto` 用于简化与 `sizeof` 相关的声明。

1.11 C语言预处理

C语言预处理（C Preprocessing）是指在C语言编译过程中的一个阶段，发生在实际编译之前。预处理器（preprocessor）负责处理源代码文件中的预处理指令。这些指令不是C语言的一部分，而是预处理器的命令，它们以井号（#）开头。

1.11.1 #include

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

用法：`#include` 指令用于包含标准库头文件或用户自定义头文件。在这个例子中，我们包含了标准输入输出库 `stdio.h`，这样我们就可以在程序中使用 `printf` 函数。

输出： Hello, world!

1.11.2 #define

```
#include <stdio.h>

#define GREET "Hello, world!"

int main() {
    printf("%s\n", GREET);
    return 0;
}
```

用法：`#define` 指令用于定义宏。这里我们定义了一个宏 `GREET`，它的值是一个字符串。

输出： Hello, world!

1.11.3 #undef

```
#include <stdio.h>

#define MESSAGE "Hello, world!"
#undef MESSAGE

int main() {
    // MESSAGE is now undefined
    return 0;
}
```

用法：`#undef` 指令用于取消宏的定义。在这个例子中，`MESSAGE` 宏首先被定义，然后被取消定义。

输出：编译错误，因为 `MESSAGE` 在 `main` 函数中被取消定义，无法使用。

1.11.4 #ifdef, #ifndef, #if, #else, #elif, #endif

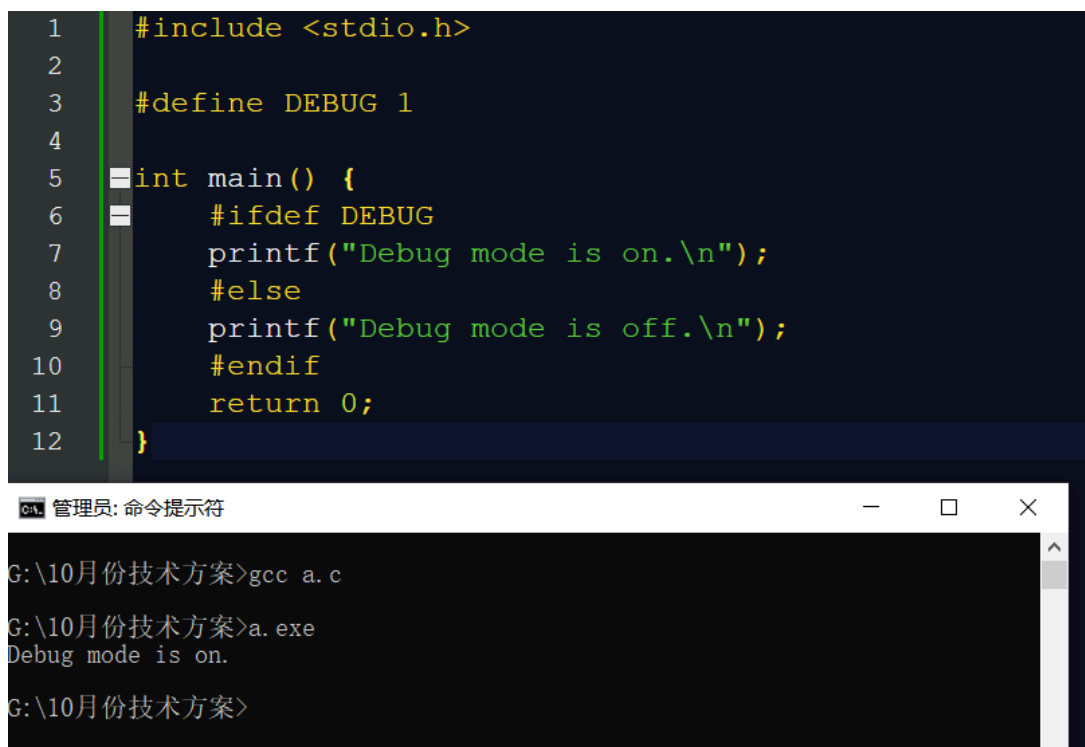
```
#include <stdio.h>

#define DEBUG 1

int main() {
    #ifdef DEBUG
        printf("Debug mode is on.\n");
    #else
        printf("Debug mode is off.\n");
    #endif
    return 0;
}
```

用法：`#ifdef` 和 `#ifndef` 指令用于条件编译。如果宏定义了，则编译 `#ifdef` 块中的代码；如果没有定义，则编译 `#else` 块中的代码。

输出：Debug mode is on.



```
1  #include <stdio.h>
2
3  #define DEBUG 1
4
5  int main() {
6      #ifdef DEBUG
7          printf("Debug mode is on.\n");
8      #else
9          printf("Debug mode is off.\n");
10     #endif
11     return 0;
12 }
```

管理员: 命令提示符

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Debug mode is on.
G:\10月份技术方案>
```

1.11.5 #pragma 这里列出

在头文件中使用 `#pragma once` 可以避免在多个源文件中包含同一个头文件时出现重复定义的问题。

```
// myheader.h
#pragma once

#define LED_ON 1
#define LED_OFF 0
```

用法：`#pragma` 指令用于提供编译器特定的指令。`#pragma once` 通常用于头文件中，以防止头文件被重复包含。

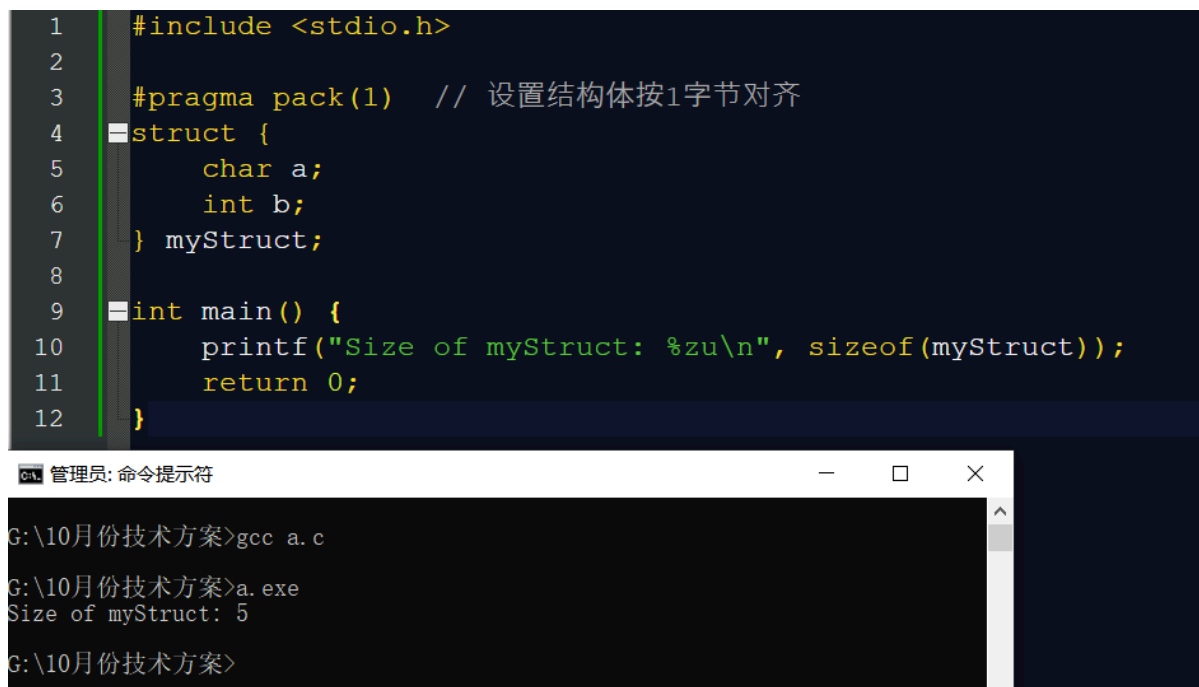
设置结构体按1字节对齐，这可以用于精确控制结构体的内存布局，以适应特定的硬件接口或协议要求。

```
#include <stdio.h>

#pragma pack(1) // 设置结构体按1字节对齐
struct {
    char a;
    int b;
} myStruct;

int main() {
    printf("Size of myStruct: %zu\n", sizeof(myStruct));
    return 0;
}
```

输出：Size of myStruct: 5



The screenshot shows a code editor with the following C code:

```
1  #include <stdio.h>
2
3  #pragma pack(1) // 设置结构体按1字节对齐
4  struct {
5      char a;
6      int b;
7  } myStruct;
8
9  int main() {
10     printf("Size of myStruct: %zu\n", sizeof(myStruct));
11     return 0;
12 }
```

Below the code editor is a Windows command prompt window titled "管理员: 命令提示符". It shows the following commands and output:

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Size of myStruct: 5
G:\10月份技术方案>
```

```
1  #include <stdio.h>
2
3  // #pragma pack(1) // 设置结构体按1字节对齐
4  struct {
5      char a;
6      int b;
7  } myStruct;
8
9  int main() {
10     printf("Size of myStruct: %zu\n", sizeof(myStruct));
11     return 0;
12 }
```



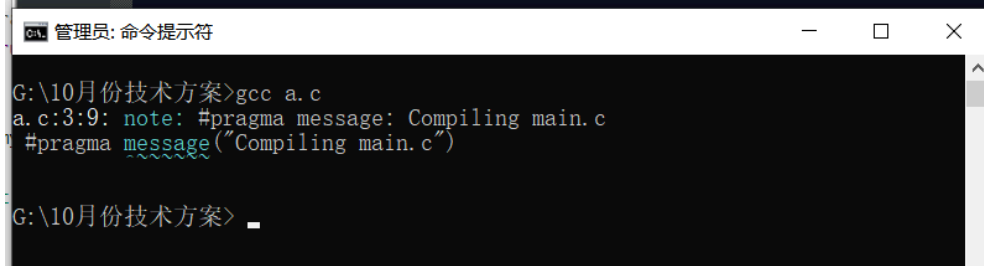
`#pragma message` 可以在编译时输出一条信息，这在STM32开发中可以用来显示编译时的状态信息或警告。

```
#include <stdio.h>

#pragma message("Compiling main.c")

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

```
1  #include <stdio.h>
2
3  #pragma message("Compiling main.c")
4
5  int main() {
6      printf("Hello, World!\n");
7      return 0;
8  }
```



`#pragma warning`

用于控制编译器的警告信息。这可以用于关闭特定的编译器警告，特别是在使用某些库或硬件抽象层时。

示例代码：

```
#pragma warning(disable : 4996) // 禁用特定警告编号

#include <stdio.h>

int main() {
    printf("This is deprecated\n"); // 这行代码会产生警告
    return 0;
}
```

在这个例子中，`#pragma warning(disable : 4996)` 用于禁用特定的编译器警告，使得后续代码中的警告不再显示。

这些 `#pragma` 指令在STM32单片机编程中非常有用，可以帮助开发者更好地控制代码的编译和优化过程。

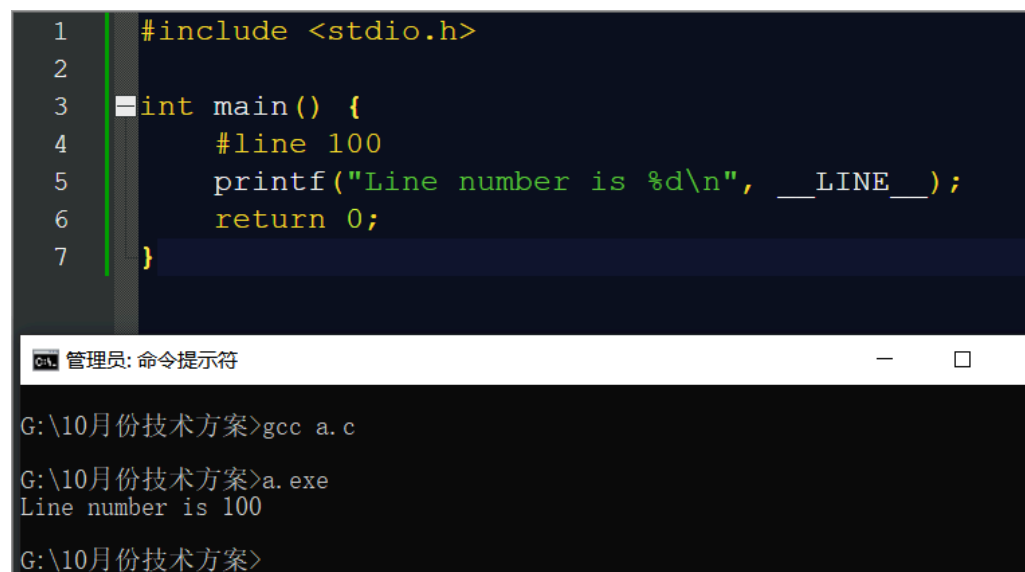
1.11.6 #line

```
#include <stdio.h>

int main() {
    #line 100
    printf("Line number is %d\n", __LINE__);
    return 0;
}
```

用法：`#line` 指令用于改变编译器报告的行号和文件名。这里我们将行号设置为 100。

输出：Line number is 100



The screenshot shows a code editor with the following C code:

```
1  #include <stdio.h>
2
3  int main() {
4      #line 100
5      printf("Line number is %d\n", __LINE__);
6      return 0;
7  }
```

Below the code editor is a Windows command prompt window titled "管理员: 命令提示符". It shows the following commands and output:

```
G:\10月份技术方案>gcc a.c
G:\10月份技术方案>a.exe
Line number is 100
G:\10月份技术方案>
```

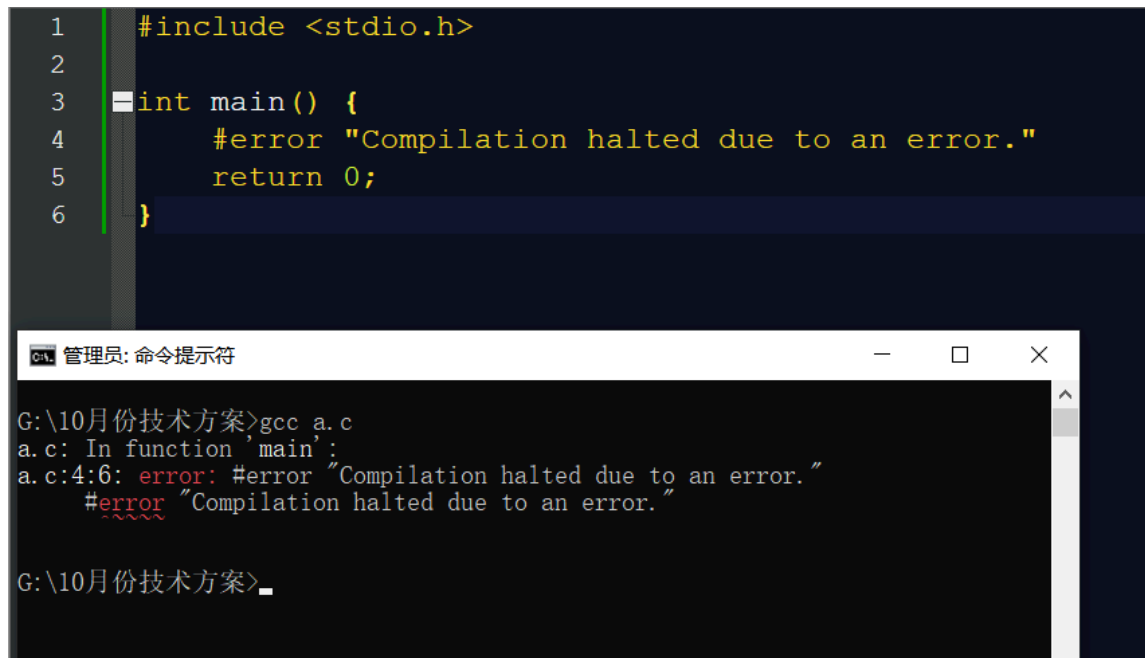
1.11.7 #error

```
#include <stdio.h>

int main() {
    #error "Compilation halted due to an error."
    return 0;
}
```

用法： `#error` 指令用于在编译时产生错误信息。

输出： 编译错误，错误信息为 `Compilation halted due to an error.`



The image shows a code editor with a C program and a Windows command prompt window. The code editor displays the following code:

```
1  #include <stdio.h>
2
3  int main() {
4      #error "Compilation halted due to an error."
5      return 0;
6  }
```

The command prompt window, titled "管理员: 命令提示符", shows the output of the compilation command `gcc a.c`:

```
G:\10月份技术方案>gcc a.c
a.c: In function 'main':
a.c:4:6: error: #error "Compilation halted due to an error."
      #error "Compilation halted due to an error."
      ~~~~~
G:\10月份技术方案>_
```

1.11.8 #warning

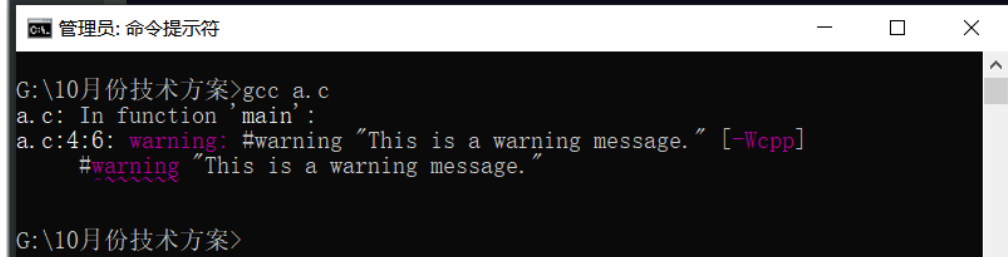
```
#include <stdio.h>

int main() {
    #warning "This is a warning message."
    printf("warning example.\n");
    return 0;
}
```

用法： `#warning` 指令用于在编译时产生警告信息。

输出： 编译警告，警告信息为 `This is a warning message.`，程序继续执行并输出 `warning example.`

```
1 #include <stdio.h>
2
3 int main() {
4     #warning "This is a warning message."
5     printf("Warning example.\n");
6     return 0;
7 }
```



The screenshot shows a Windows command prompt window titled "管理员: 命令提示符". The user is in the directory "G:\10月份技术方案". They have compiled a C program named "a.c" using "gcc a.c". The output shows the compiler's internal processing of the #warning directive, indicating it is a warning and that the warning message will be printed. The warning is shown as "#warning: #warning 'This is a warning message.' [-Wcpp]" and the message itself is "#warning 'This is a warning message.'".

这些示例展示了 C 语言预处理指令的基本用法和效果。预处理指令在编译过程中的第一阶段处理，它们不会影响程序的执行逻辑，但会影响编译过程和最终生成的代码。

2. 指针篇

指针是什么

指针和变量

指针数组

指针函数

数组指针

函数指针-回调函数

指针和二维数组

二级指针

指针和结构体

3. 高频出现的编程题篇

冒泡排序

字符串拷贝

字符串查询

链表逆序

4. 其他

补码

反码

结构体大小计算

联合体大小计算

结构体位

大小端字节序

二. STC51单片机阶段

1. 学习定位总结

2. 定时器总结

3. UART总结

4. IIC总结

5. 常用外设模块总结

三. Linux系统编程阶段

1. 为什么要用Linux系统

2. 文件总结

3. 进程总结

4. 进程间通信总结和使用场景

5. 线程总结

6. 线程同步总结

7. 网络编程总结

8. 第三方库使用总结

四. 全志Linux阶段

1. 为什么用高级ARM

2. 什么是BSP，难度定位

3. ARM+Linux的外设和单片机有何区别

4. 设备驱动总结

5. 图像垃圾桶总结和技术定位

6. 智能家居总结及技术定位

五. STM32阶段

1. 常用外设通信总线

2. FreeRTOS总结

3. RTthread总结

六. C++QT阶段

1. C++高频笔试题

2. 信号与槽总结

3. 事件处理总结

4. 自定义控件总结

5. C++QT的定位

七. 项目描述
